

Informe 3: soluciones innovadoras con Arquitecturas de Microservicios

Integrantes:

- Nelson Cofré
- Nicolas Sanchez
- Mario Cofré

Docente:

- Claudio Rojas

0. Introducción

Donaton es una plataforma de ayuda humanitaria que centraliza donaciones, necesidades en terreno y logística de distribución. Se implementó con arquitectura de microservicios, BFF, API Gateway (KrakenD), frontend React, backend Spring Boot y despliegue en Kubernetes.

El sistema está operativo de punta a punta: el usuario accede por el frontend, las peticiones pasan por KrakenD y el BFF, y orquesta cuatro microservicios con bases PostgreSQL independientes. Desarrollamos 90 pruebas automatizadas (backend + frontend) y documentamos arquitectura, APIs, despliegue y testing en el repositorio.

Además, dado el impacto social de la solución, la arquitectura debe favorecer la **transparencia**, la **trazabilidad** y una **distribución más equitativa** de los recursos.

1. SECCIÓN 1 — Diseño de arquitectura y patrones

1.1 Problema y objetivo

En emergencias, la coordinación informal (redes sociales, hojas de cálculo) genera duplicación de recursos y desabastecimiento en otras zonas. Donaton busca transparentar el flujo desde la donación hasta la entrega final.

Objetivo arquitectónico: diseñar una solución modular, escalable y desacoplada que separe donaciones, necesidades y logística en dominios independientes.

1.2 Actores del caso

Donantes individuales / Empresas

- Rol en el sistema: Registran donaciones

Centro investigador (municipalidades)

- Rol en el sistema: Reportan necesidades con ubicación

Almacenes (centros de acopio)

- Rol en el sistema: Almacenan recursos

Transporte (equipos logísticos)

- Rol en el sistema: Gestionan envíos y entregas

1.3 Arquitectura propuesta e implementada

Usuario → Frontend (React) → API Gateway (KrakenD) → BFF → Microservicios → PostgreSQL

Frontend

- Responsabilidad: Interfaz web

API Gateway

- Responsabilidad: Enrutamiento, CORS, rate limit

BFF

- Responsabilidad: Orquestación, adaptación de DTOs, validación JWT

auth-service

- Responsabilidad: Usuarios, login, JWT

donation-service

- Responsabilidad: Donaciones y donantes

necessity-service

- Responsabilidad: Necesidades en terreno

logistics-service

- Responsabilidad: Centros, inventario, envíos

Patrón Database per Service: cada microservicio tiene su propia base PostgreSQL; no hay relaciones directas entre bases, solo comunicación HTTP.

1.4 Microservicios clave

Autenticación

- Entidades principales: Usuario, Rol (ADMIN/USER), Token

Donaciones

- Entidades principales: Donante, Recurso, Donación

Necesidades

- Entidades principales: Necesidad, Ubicación, Prioridad, Estado

Logística

- Entidades principales: CentroAcopio, Inventario, Envío
-

1.5 API Gateway

KrakenD centraliza el acceso del frontend al backend. Enruta hacia el BFF y aplica políticas transversales (CORS, rate limit en login). La validación JWT la realiza el BFF.

1.6 Patrones de diseño seleccionados

Repository

- Aplicación: Acceso a datos con Spring Data JPA
- Estado: Implementado

Factory Method

- Aplicación: Creación polimórfica de recursos
- Estado: Diseñado; pendiente en código

BFF

- Aplicación: API consolidada para el frontend
- Estado: Implementado

DTO + Mapper

- Aplicación: Adaptación de contratos en el BFF
 - Estado: Implementado
-

1.7 Escalabilidad, desacoplamiento y ética

- Escalabilidad: cada servicio escala según su carga (p. ej. más reportes de necesidades sin escalar donaciones).
 - Desacoplamiento: cambios internos no afectan otros servicios si se mantiene el contrato HTTP.
 - Ética: transparencia del flujo donación → entrega, priorización por necesidades reales y protección de datos sensibles (ubicación, requerimientos de comunidades).
-

2. SECCIÓN 2 — Desarrollo frontend y backend

2.1 Stack tecnológico

Frontend

- Tecnología: React 19, TypeScript, Vite, React Router

Backend

- Tecnología: Java 21, Spring Boot 3, Gradle

Persistencia

- Tecnología: PostgreSQL, JPA, Flyway

Comunicación

- Tecnología: REST, WebClient (BFF)

Contenedores

- Tecnología: Docker, Kubernetes

Documentación API

- Tecnología: Swagger / OpenAPI
-

2.2 Backend For Frontend (BFF)

El BFF es el punto de orquestación entre frontend y microservicios:

- Expone rutas unificadas (/api/auth/*, /api/donations, /api/v1/necessities, /api/v1/logistics/*).
 - Adapta DTOs del frontend al formato de cada microservicio (mappers).
 - Válida JWT en rutas protegidas.
-

2.3 Microservicios con JPA

Se implementaron cuatro microservicios con entidades JPA, repositorios y migraciones Flyway:

- Creación automática de esquema al iniciar.
 - Datos semilla en necesidades (10 registros de ejemplo, emergencia Valparaíso).
 - Perfil test con H2 en memoria para pruebas sin PostgreSQL.
-

2.4 Frontend

Organizado con Feature-Sliced Design (capas: app → pages → widgets → features → entities → shared).

Módulos funcionales: login/registro, donaciones, necesidades, logística (centros, inventario, envíos).

Autenticación: JWT en localStorage; rutas protegidas con patrón Protected Route.

Módulos NPM: la reutilización se logra mediante slices con API pública (index.ts) e imports modulares (@/entities/donation), sin publicar paquetes externos.

2.5 Integración frontend ↔ backend

Registro e inicio de sesión

- Estado: ✓

CRUD donaciones

- Estado: ✓

CRUD necesidades

- Estado: ✓

CRUD logística

- Estado: ✓

API Gateway (CORS, enrutado)

- Estado: ✓

URLs de despliegue (Kubernetes): Frontend <http://localhost:30517> · Gateway <http://localhost:30090>

2.6 Estrategia de branching - Git Flow

Rama main

- Uso: Versión estable

Rama development

- Uso: Integración del equipo

Rama feature/*

- Uso: Desarrollo por funcionalidad

Flujo: feature → Pull Request → development → merge a main cuando esté estable.

3. SECCIÓN 3 — Integración, pruebas y cierre

3.1 Integración del sistema

Todos los módulos se integran en un flujo único:

1. Usuario opera en el frontend.
 2. Petición HTTP al API Gateway.
 3. KrakenD enruta al BFF.
 4. BFF válida JWT y llama al microservicio correspondiente.
 5. El microservicio persiste en su PostgreSQL y retorna la respuesta.
-

3.2 Estrategia de pruebas

90 Pruebas (documentadas en docs/TESTING.md, ejecutables con scripts/test-all.ps1).

ms-auth

- Tests: 12
- Tipo: JWT unitario + API integración

ms-donation

- Tests: 12
- Tipo: Servicio + CRUD API

ms-necessity

- Tests: 11
- Tipo: Servicio + CRUD API

ms-logistic

- Tests: 8
- Tipo: CRUD logística

bff

- Tests: 28
- Tipo: Mappers, filtro JWT

frontend

- Tests: 19
- Tipo: Componentes UI, auth, mappers

Herramientas: JUnit 5, Mockito, MockMvc, H2, JaCoCo (backend) · Vitest, Testing Library (frontend).

Tipos de prueba: unitarias (lógica de negocio), integración API (CRUD completo), BFF (mappers y seguridad), frontend (componentes y utilidades).

3.3 Cobertura y calidad

- **JaCoCo** genera reportes HTML/XML por módulo backend.
 - **Vitest** genera cobertura del frontend (frontend/coverage/).
 - El caso exige **≥60% con SonarQube**: la medición local está lista; la integración con SonarQube y el pipeline CI automático quedan como mejora pendiente.
-

3.4 Refactorización aplicada

- Extracción de orquestación al BFF con mappers por dominio.
 - Migraciones Flyway en lugar de esquemas manuales.
 - Reorganización del frontend con FSD.
 - Manejo centralizado de errores (Exception Handler).
 - Filtros JWT transversales (sin repetir lógica en cada endpoint).
-

3.5 Despliegue

```
.\scripts\build-k8s-images.ps1 # Construir imágenes
.\scripts\deploy-k8s.ps1      # Desplegar en K8s
kubectl get pods -n donaton    # Verificar estado
```

4. Conclusión

La arquitectura propuesta para Donaton se basa en una separación clara de dominios mediante microservicios: gestión de donaciones, gestión de necesidades, gestión logística y autenticación. Esta decisión permite responder de mejor forma a los problemas de coordinación identificados en el contexto del caso, mejorando la organización interna del sistema, facilitando el mantenimiento y permitiendo escalabilidad independiente.

La incorporación de una API Gateway (KrakenD) centraliza el acceso al backend y mejora la comunicación con el frontend, mientras que el BFF orquesta las peticiones hacia cada microservicio. La aplicación del Repository Pattern, tanto en backend (Spring Data JPA) como en frontend, fortalece la calidad técnica, la extensibilidad y el desacoplamiento de la solución. En conjunto, esta propuesta constituye una base sólida para una plataforma tecnológica escalable, desacoplada y alineada con el propósito social de Donaton.

El frontend de la plataforma funciona con React y se comunica con el backend a través del API Gateway y el BFF, sin acceder directamente a los microservicios. Cada microservicio trabaja con su propia base de datos (database per service), apoyándose en repositorios Spring Data JPA y migraciones Flyway para la persistencia.

Para garantizar la eficacia del equipo y gestionar las versiones del proyecto, utilizamos GitHub con una estrategia de branching inspirada en Git Flow, con las ramas main y development, y ramas de trabajo feature/* y refactor/*.

Junto a esto, cada microservicio utiliza Docker para funcionar en un entorno comprobado, con dependencias consistentes, de forma que el sistema se comporte de manera uniforme en distintos equipos y servidores.

Junto a esto, el stack completo puede desplegarse en Kubernetes para orquestar los contenedores de forma escalable y uniforme en distintos equipos y servidores.

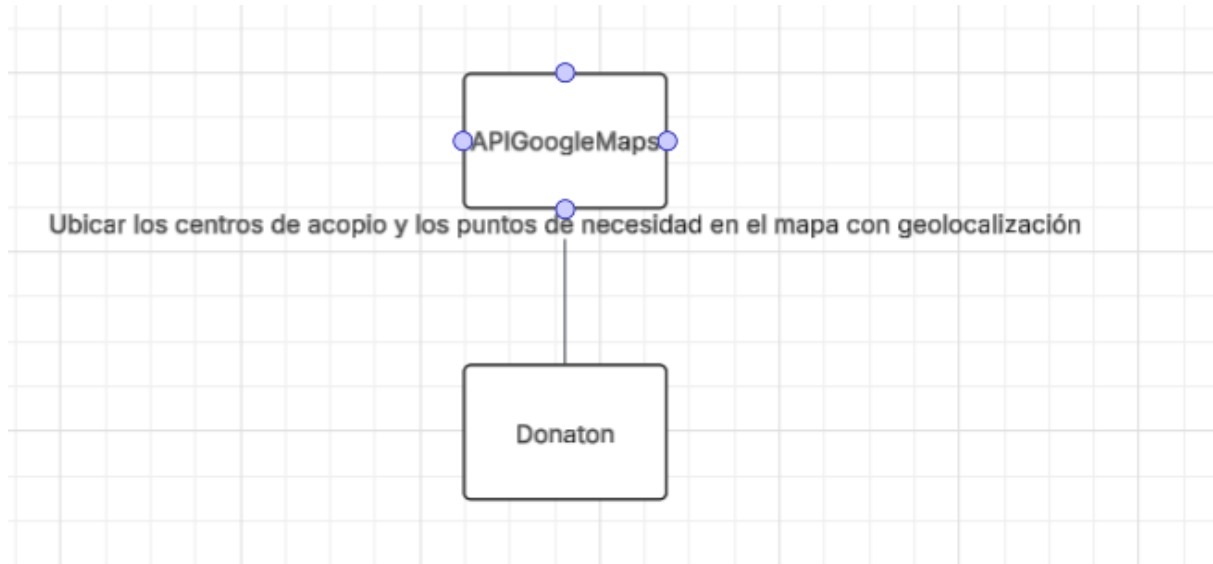
El proyecto está optimizado para funcionar de punta a punta (integración end-to-end): partiendo desde el frontend con React, según la interacción del usuario; después pasando por el API Gateway (KrakenD) y el BFF; entregando la información al microservicio correspondiente; y almacenando los datos en la base de datos respectiva de cada servicio.

Realizamos ~90 tests para comprobar el funcionamiento del proyecto, desde operaciones CRUD y validación JWT hasta componentes de la interfaz de usuario.

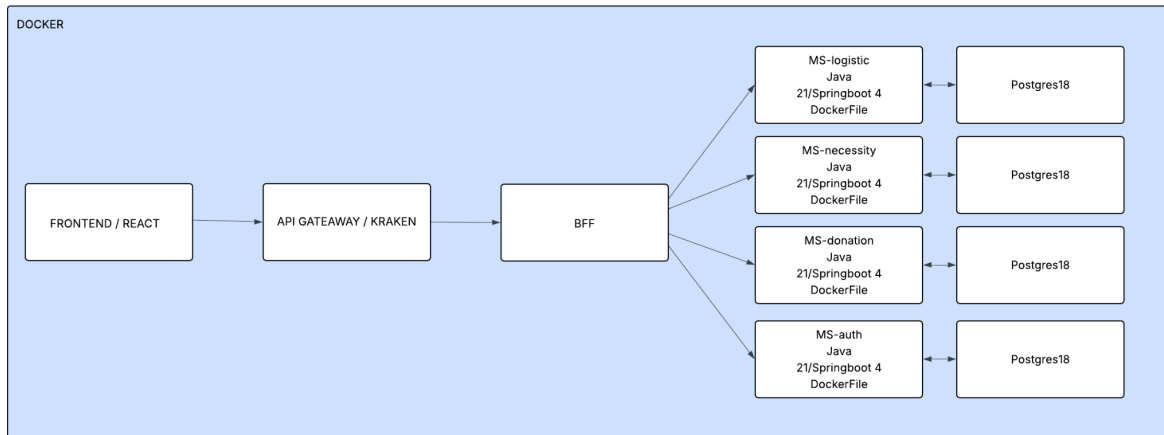
Finalmente, para un desarrollo ordenado, organizamos el frontend y el backend con separación clara de responsabilidades —Feature-Sliced Design en React y capas por microservicio en Spring Boot—, reduciendo confusiones y posibles errores derivados de un diseño poco estructurado.

Diagramas C4: Contexto

Link: [Enlace Diagramas C4](#)



Diagramas C4: Contenedor



Estructura del Frontend:

```
src/
├─ app/                                # Configuración global de la aplicación
│   ├─ App.tsx                        # Router principal
│   ├─ layouts/
│   │   └─ MainLayout.tsx            # Layout con navbar y Outlet
│   ├─ providers/
│   │   └─ AuthProvider.tsx          # Contexto de autenticación
│   └─ routes/
│       └─ ProtectedRoute.tsx         # Guard de rutas privadas
│
├─ assets/                             # Imágenes, iconos, fuentes estáticas
│
├─ config/                             # Configuración global
│   └─ env.ts                         # VITE_API_BASE_URL, etc.
│
├─ features/                           # Módulos por dominio de negocio
│   ├─ auth/
│   ├─ donations/
│   ├─ needs/
│   └─ logistics/
```

[illegible]